

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №2 (Яндекс.Контест)

Выполнил:

Студент группы Р3231

Тимошкин Р.В.

Преподаватель:

Косяков М.С.

Санкт-Петербург, 2025г.

Задача №5 «Е. Коровы в стойла»

Описание:

К коров расставлены в N упорядоченных стойлах так, чтобы минимальное расстояние между ними было как можно большее. Нужно найти такое расстояние.

Доказательство:

Для решения задачи подберём расстояние между стойлами так, чтобы разместить всех коров с максимальным расстоянием, используя бинарный поиск в интервале от 0 до расстояния между первой и последней координатой. На каждом шаге проверяем текущее расстояние, расставляя коров, начиная с первого стойла, и сравниваем количество размещённых коров с требуемым. Если все коровы помещаются, увеличиваем левую границу, иначе уменьшаем правую, пока границы не совпадут, что и будет искомым решением.

Алгоритмическая сложность:

- По времени - $O(n \cdot \log(dist))$, где $dist$ - расстояние между первым и последним стойлом.
- По памяти: - $O(n)$.

Код:

```
1. #include <cstdint>
2. #include <stdint>
3. #include <iostream>
4. #include <vector>
5.
6. namespace {
7. inline auto PlaceCows(const std::vector<uint64_t>& stalls, size_t
   from, size_t cows) -> bool {
8.     size_t placed = 1;
9.     uint64_t prev = stalls.front();
10.
11.     for (size_t i = 1; i < stalls.size(); ++i) {
12.         if (stalls.at(i) - prev >= from) {
13.             ++placed;
14.             prev = stalls.at(i);
15.         }
16.     }
17.
18.     return placed >= cows;
19. }
20.
21. auto FindMaxDistance(const std::vector<uint64_t>& stalls, size_t
   cows) -> size_t {
22.     size_t max_distance = 0;
23.
24.     size_t first = 1;
25.     size_t last = stalls.back() - stalls.front() + 1;
26.
27.     while (first <= last) {
28.         size_t middle = (last + first) / 2;
29.
```

```

30.         if (PlaceCows(stalls, middle, cows)) {
31.             max_distance = middle;
32.             first = middle + 1;
33.         } else {
34.             last = middle - 1;
35.         }
36.     }
37.
38.     return max_distance;
39. }
40. } // namespace
41.
42. auto main() -> int {
43.     size_t stall_cnt = 0;
44.     size_t cows_cnt = 0;
45.
46.     std::cin >> stall_cnt;
47.     std::cin >> cows_cnt;
48.
49.     std::vector<uint64_t> stalls(stall_cnt);
50.     for (size_t i = 0; i < stall_cnt; ++i) {
51.         std::cin >> stalls[i];
52.     }
53.
54.     auto max_distance = FindMaxDistance(stalls, cows_cnt);
55.     std::cout << max_distance << "\n";
56. }

```

Задача №6 «F. Число»

Описание:

Из нескольких строк (хотя бы в одной строке первая цифра отлична от нуля) нужно собрать одну строку, которая представляется максимальным числом из всех возможных способов сбора строк.

Доказательство:

Данное решение работает корректно, так оно сортирует строки с цифрами таким образом, чтобы их объединение давало максимальное возможное число. Это достигается с помощью компаратора, который сравнивает две строки a и b по результату их конкатенации в обоих порядках ($a+b$ и $b+a$). После сортировки строки объединяются в результирующее число.

Алгоритмическая сложность:

- По времени - $O(n \cdot \log(n))$.
- По памяти: - $O(n)$.

Код:

```

1. #include <algorithm>
2. #include <iostream>
3. #include <numeric>
4. #include <string>

```

```
5. #include <vector>
6.
7. namespace {
8. auto CompareForMaxNumber(const std::string& first, const std::string&
   second) -> bool {
9.     return first + second > second + first;
10. }
11.
12. auto FindMaxNumber(std::vector<std::string>& pieces) ->
   std::string {
13.     std::ranges::sort(pieces, CompareForMaxNumber);
14.
15.     return std::reduce(
16.         pieces.begin(),
17.         pieces.end(),
18.         std::string{},
19.         [](const std::string& acc, const std::string& piece) {
   return acc + piece; }
20.     );
21. }
22. } // namespace
23.
24. auto main() -> int {
25.     std::vector<std::string> pieces;
26.     std::string input;
27.
28.     while (std::cin >> input) {
29.         pieces.emplace_back(input);
30.     }
31.
32.     auto max_number = FindMaxNumber(pieces);
33.
34.     std::cout << max_number << '\n';
35.
36.     return 0;
37. }
```